IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2023

INSTRUCTION ARCHITECTURES AND COMPILERS

Friday 12 May 14:30

Time allowed: 2 hours

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible	First Marker(s) :	J. Wickerson,
	Second Marker(s) :	P. Cheung

© Imperial College London

INSTRUCTION ARCHITECTURES AND COMPILERS

NB: Questions 1–5 relate to the Instruction Architectures half of the module. Questions 6–10 relate to the Compilers half of the module. A summary of the RISC-V RV32I instruction set is provided at the back of this paper.

1. Use the instruction set summary provided, translate the following RISC-V RV32I assembly language instructions into machine code instruction as 32-bit hexadecimal numbers.

   a) `lw s0, 255(t1)` [3]

   b) `xor a0, t1, t2` [3]

2. What are the assembly language instructions of the following RISC-V RV32I machine code instructions?

   a) `16'h000302b3` [4]

   b) `16'h00a12523` [4]

3. Figure 3.1 shows the design of a finite state machine `fsm` in SystemVerilog. The state machine has one input signal `x` and one output signal `z`. The state machine is controlled by the clock signal `clk`.

a) How many states are there? [1]

b) How is the state machine initialised? [1]

c) Draw the state diagram of this state machine. [4]

d) Complete the timing diagram shown in Figure 3.2 including the state of `fsm` at clock cycles T0 to T9. You may assume that the `rst` signal is low throughout, the input signal `x` changes shortly before the rising edge of the clock signal, and that `fsm` is initially at state `A`. To help you show your answer, a duplicate copy of Figure 3.2 is provided at the end of this paper. You can tear it off, annotate the diagram and attach it to the answer book. [4]

```
module fsm (
    input  logic    clk,    // clock
    input  logic    rst,    // reset state machine
    input  logic    x,      // input signal x
    output logic    z       // output signal z
);

    // define states
    typedef enum {A, B, C, D} my_states;
    my_state current_state, next_state;

    // state registers
    always_ff @(posedge clk)
        if (rst)    current_state <= A;
        else        current_state <= next_state;

    // next state logic
    always_comb
        case (current_state)
            A: if (x) next_state = B;
               else next_state = A;
            B: if (x) next_state = C;
               else next_state = A;
            C: if (x) next_state = A;
               else next_state = D;
            C: if (x) next_state = A;
               else next_state = B;
        default: next_state = current_state;
        endcase

    // output logic
    always_comb
        case (current_state)
            A: z = 0;
            B: z = 0;
            C: z = 1;
            D: z = 0;
        default: Z = 0;
        endcase
endmodule
```

Figure 3.1

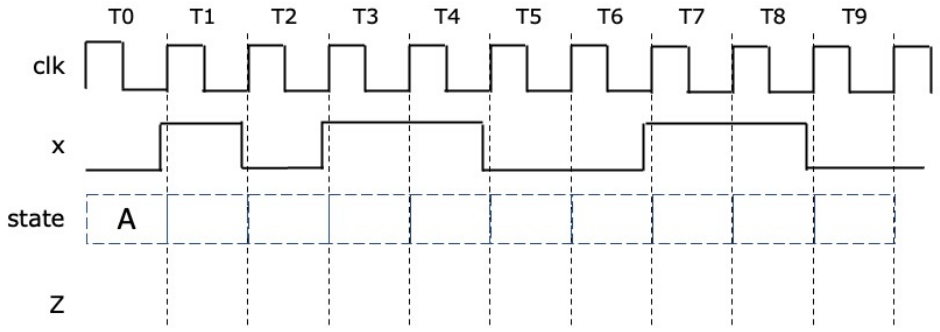


Figure 3.2

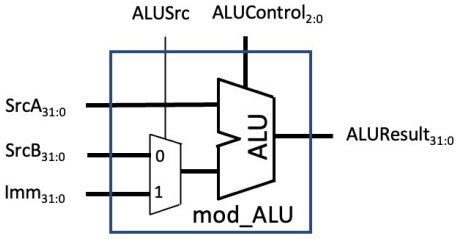
4. Figure 4.1 shows the module `mod_ALU` that implements the ALU hardware for the following five RISC-V RV32I instructions:

- `add rd, rs1, rs2`
- `sub rd, rs1, rs2`
- `sll rd, rs1, rs2`
- `addi rd, rs1, imm`
- `slti rd, rs1, imm`

The meanings of the `ALUControl[2:0]` signals for the four operations are also shown in Figure 4.1. You are responsible for the design of this `mod_ALU` module only.

a) Design the module `mod_ALU` (only this module) in SystemVerilog to execute these four instructions. [7]

b) Explain with the aid of a diagram how the bits in the 32-bit instruction can be used to produce `ALUSrc` and `ALUControl[2:0]` signals without any logic gates. [5]



ALUControl$_{2:0}$	Function
000	add
001	shift left logical
010	set less than
100	subtract

Figure 4.1

5. Figure 5.1 shows an implementation of the RISC-V RV32I processor using five stages of pipelining when executing four instructions.

a) Explain what each of the pipeline stages does. [4]

b) What are data hazards in a pipelined CPU architecture? Briefly describe two techniques that can be used to mitigate such hazards. [4]

c) For the four instructions shown in Figure 5.1, identify where and when data hazards occur. To help you show your answer, a duplicate copy of Figure 5.1 is provided at the end of this paper. You can tear it off, annotate the diagram and attach it to the answer book. [6]

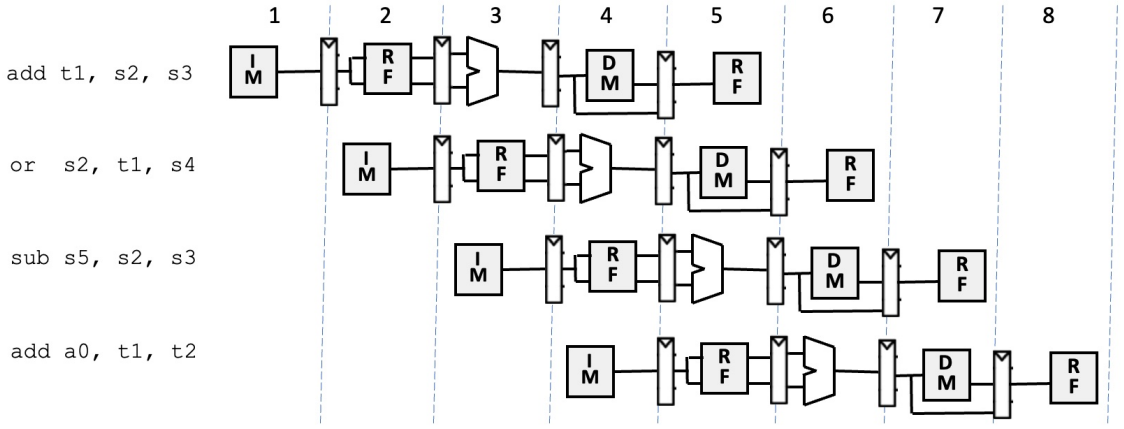


Figure 5.1

6. An application can be linked with a code library either statically (at compile time) or dynamically (at run time).

   a) Provide one reason why static linking could lead to better run-time performance than dynamic linking. [3]

   b) Provide one reason why dynamic linking could lead to better run-time performance than static linking. [3]

7. A C compiler can support '`do...while`' loops by transforming them into `while` loops. Briefly describe this transformation, and explain why it may not be the most efficient compilation strategy. [6]

8. Consider the following C function.

```
int foo (int x) {
  return (x+1 < x);
}
```

   Explain how a compiler could optimise this function by exploiting the concept of 'undefined behaviour'. [6]

9. Recall that (basic) regular expressions are constructed using the following grammar, in which c ranges over the symbols in the alphabet.

$$r ::= 0 \mid 1 \mid c \mid r + r \mid rr \mid r^* \mid (r)$$

   Throughout this question, let us use $\{\mathtt{a}, \mathtt{b}\}$ as our alphabet.

   a) Explain two ways in which a non-deterministic finite automaton (NFA) can exhibit non-determinism. [4]

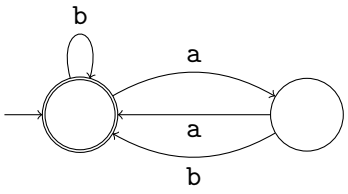
   b) Sketch an NFA that accepts the same language as the following regular expression.

$$(\mathtt{a}(1 + \mathtt{b}))^*$$

   [Hint: Your NFA should not require more than two states.] [6]

   c) With the help of the Rabin–Scott subset construction or otherwise, sketch an equivalent *deterministic* finite automaton (DFA). [5]

   d) Write down a regular expression that accepts the same language as the following DFA. [4]



   e) Define the *length* of a regular expression as the number of (non-whitespace) characters it contains. For example, the length of the regular expression

$$(\mathtt{a}(1 + \mathtt{b}))^*$$

   is 9. If I have a regular expression of length N, is it the case that for every $M > N$ there exists a regular expression of length M that accepts the same language? Explain your answer. [5]

10. Consider the following type-inference rule for if-expressions.

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}$$

a) Explain why the rule *cannot* be used to infer a type for the following expression.

```
if false then 5 else true
```

[3]

b) Propose a new rule that, if added to the existing set of rules, would allow a reasonable type to be inferred for the expression above. [5]